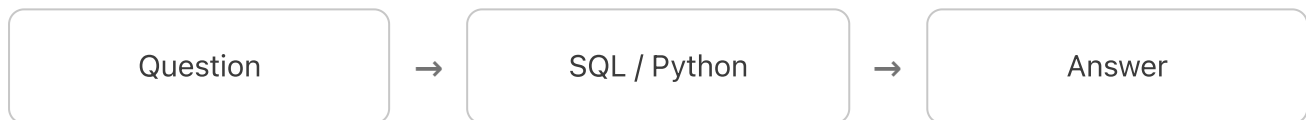


Inside Codex Data

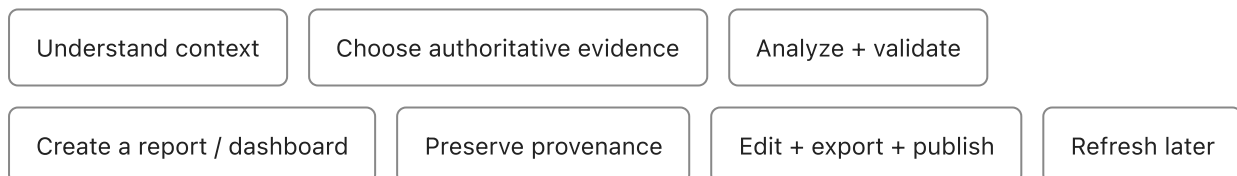
How an analytics agent turns a question into evidence you can inspect, an artifact you can edit, and work you can revisit.

What if the job continues after the answer?

THE SIMPLE LOOP



THE DURABLE ANALYTICAL PRODUCT



Codex Data is designed for this larger job.

— Under the Hood / implementation + sources

ARCHITECTURE DETAIL • INTERPRETATIONS IDENTIFIED IN CONTEXT

The plugin combines analytical instructions with an artifact platform. The host performs source execution. Data's report and dashboard contracts preserve reviewed evidence, common presentation behavior, and delivery state. The simple loop is a teaching contrast, not a claim about another product.

Agent → Skills → Evidence → Artifact

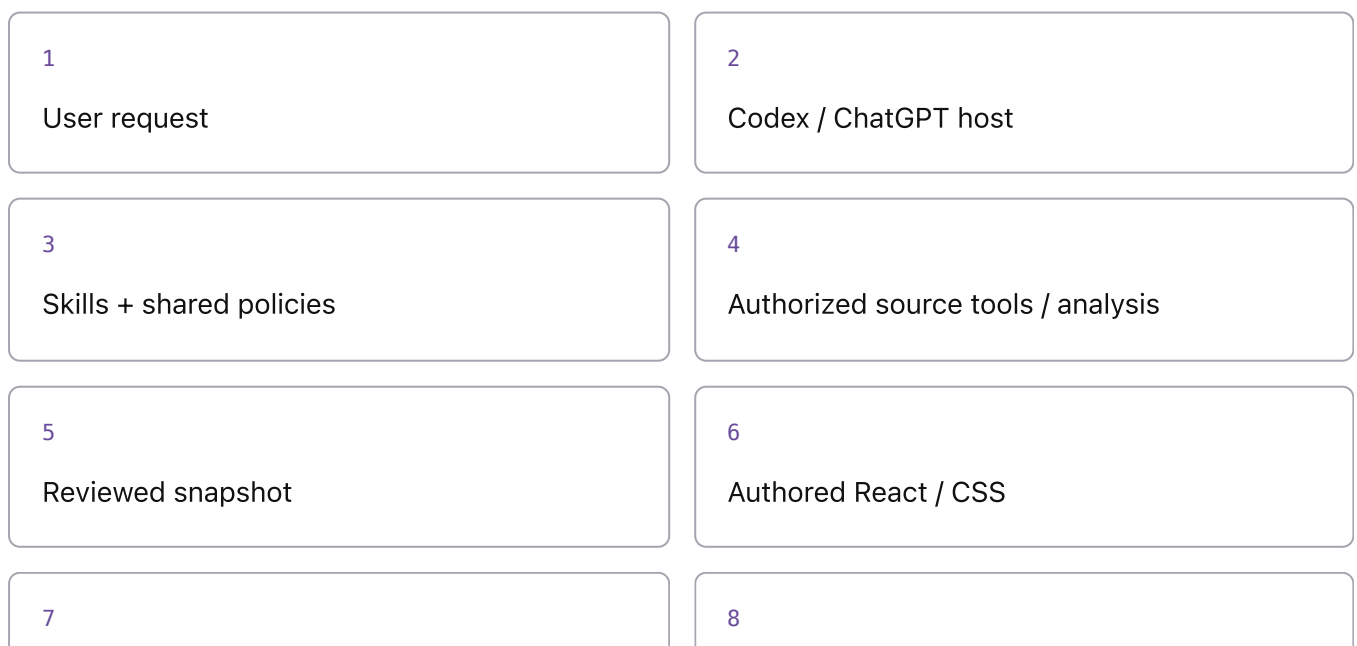
The agent does the work. Data gives that work responsibilities, evidence boundaries, and a durable form.



INSIDE EVIDENCE

Reviewed query rows travel with source requests, execution time, methods, definitions, grain, and limitations. The contract asks the agent to preserve this information; metadata alone does not prove correctness.

[Return to four boxes ↑](#)



Verified build + shared runtime

Report / dashboard → export / Site

Notebook and document workflows branch to their own artifact tools; they do not all run through the React app.

The host supplies execution. Data supplies the analytical workflow and artifact contract.

— Under the Hood / implementation + sources

ARCHITECTURE DETAIL · INTERPRETATIONS IDENTIFIED IN CONTEXT

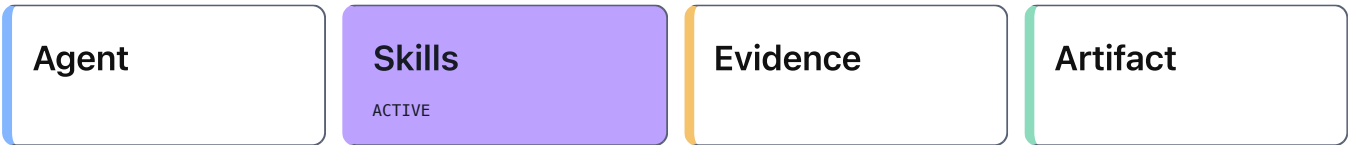
This is an architectural interpretation of the implementation, not a literal four-process pipeline. Skills are text instructions consumed by the agent. The shared React runtime is the report/dashboard path; notebook creation and document/slide conversion have separate tool dependencies.

Follow one request.

“Why did advisor engagement decline last quarter? Build me a dashboard explaining the drivers.”

Hypothetical request · illustrative route, not an executed investigation

ACTIVE RESPONSIBILITY



Steps may iterate as evidence develops. The selected dashboard output remains the same.

STEP 1 / INDEX → BUILD-DASHBOARD

Interpret the request

Lock the requested output first. The source might be a warehouse, a governed report, or supplied files; source format does not select the deliverable.

INPUT	PRODUCES
“Why” + “last quarter” + “dashboard”	Investigation intent · dashboard output

— Under the Hood / implementation + sources

ARCHITECTURE DETAIL · INTERPRETATIONS IDENTIFIED IN CONTEXT

The index selects requested output before source-helper execution. The walkthrough is one plausible route grounded in the skills, not a mandatory call trace. Diagnostics, context gathering, quality checks, and authoring can iterate. No advisor data was retrieved and no driver conclusion is asserted here.

Skills are responsibilities. Tools are capabilities.

“Define success,” “diagnose a movement,” and “check the conclusion” are different jobs—even if each can use SQL.

Understand
Frame the question and its meaning.

Analyze
Choose the right analytical job.

Validate
Challenge the evidence and the conclusion.

Create
Compose the experience the reader needs.

Deliver
Carry an existing artifact into its next use.

publish-artifact-to-sites

report-to-pdf

convert-to-doc

convert-to-slides

share-artifact-summary

schedule-refresh-jobs

DELIVER / RESPONSIBILITY
schedule-refresh-jobs
Set up a recurring cloud refresh for an existing app.

WHEN INVOKED	A request to keep a dashboard or report up to date.
NEEDS	Stable app/Site identity, rereadable sources, cadence, timezone, access.
OWNS	Creating or updating and verifying the cloud automation.
BOUNDARY	Executing a one-time refresh or silently substituting a local cron job.

TWO SUMMARIZED RULES

- Match the app by verified identity, not title alone.
- Each run refreshes the app; it does not create another schedule.

Common companions / handoffs
build-dashboard · build-report
Context dependent, not a fixed execution graph.

► Example request

[Inspect skill source ↗](#)

A distinction worth remembering

Create artifact ≠ Publish artifact



Create artifact

Author and validate a useful analytical experience.



Publish artifact

Deliver the exact reviewed experience to a destination.

— [Under the Hood](#) / [implementation + sources](#)

ARCHITECTURE DETAIL · INTERPRETATIONS IDENTIFIED IN CONTEXT

The 20 installed entry points form a procedural instruction graph. This explorer summarizes the skill contracts. “Common companions” means context-dependent workflow relationships, not guaranteed automatic calls. Shared policies own source authority and dependency handling; the index retains final-output ownership.

Preserve the basis of the answer.

Raw data tells you what a field contains. Reviewed evidence also tells you what it means, where it came from, and what limits apply.



Inspection follows the binding backward.

The reader can inspect saved definitions, rows, SQL, and evidence flow. This source view does not automatically rerun the original query.

Illustrative teaching fixture

18.4%

184 engaged / 1,000 eligible

This demonstrates the contract. It is not advisor research.

rows

source

executedAt

methods

definitions

coverage

What could be missing?

Record pagination, truncation, missingness, and cohort maturity where relevant. A saved result is only as complete as its extraction.

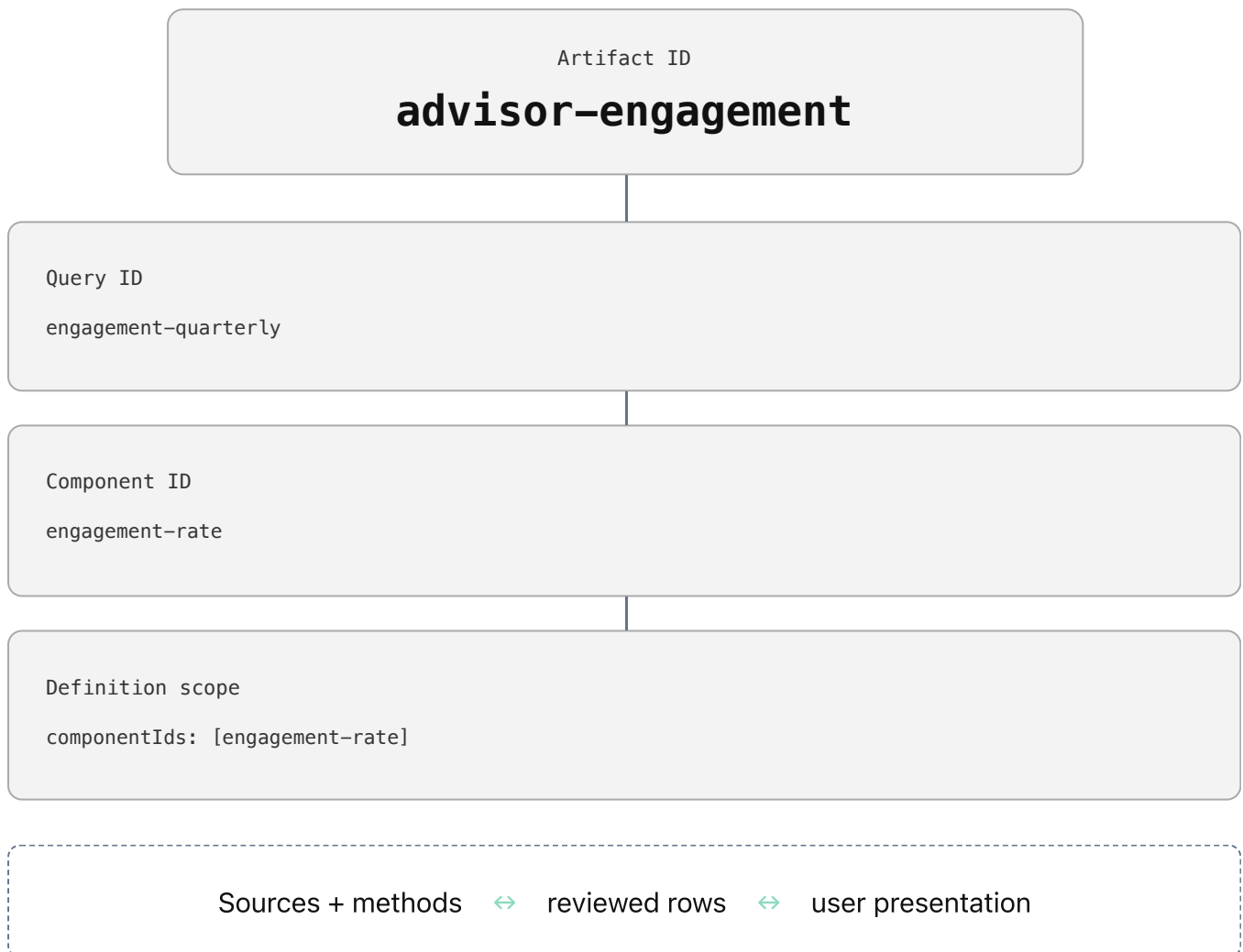
Provenance travels with the result, rather than depending on the conversation.

ARCHITECTURE DETAIL · INTERPRETATIONS IDENTIFIED IN CONTEXT

Real query data lives under stable keys in `src/data.json`. SQL lives in `source.sql`; successful non-secret tool requests can be recorded in `source.evidenceFlow`; transformations belong in `query.methods`. Metric definitions can be scoped to component IDs. Coverage and units are contractual metadata, not necessarily top-level keys. Source inspection displays saved evidence; it does not imply a rerun.

Give important objects a stable identity.

A refresh should update the evidence—not make every chart, saved edit, and source reference a different object.



Illustrative refresh

Snapshot B · 17.1% · next period · IDs preserved

The value and timestamp change. The logical identities stay put.

ARCHITECTURE DETAIL · INTERPRETATIONS IDENTIFIED IN CONTEXT

Preparation creates a new artifact identity and rejects an existing destination. Revisions preserve app/query/component IDs. DataComponent requires a non-empty stable ID. Identity gives references continuity; it is not a content hash, authorization token, or proof that evidence is unchanged.

Edit the presentation. Preserve the evidence.

Rows, sources, and methods live in one state domain. Titles, layout, hidden blocks, and chart options live in another.

EVIDENCE STATE · TEACHING FIXTURE

```
{  
  "queryId": "engagement-quarterly",  
  "engaged": 184,  
  "eligible": 1000,  
  "rate": "18.4%",  
  "source": "illustrative teaching fixture"  
}
```

Stored evidence is unchanged by presentation edits.

Advisor engagement

18.4%

Illustrative reviewed metric

Narrative: "Engagement is 18.4%."

Filters can change which stored rows are visible. Hiding a block or filtering a view does not revoke access to the underlying snapshot.

ARCHITECTURE DETAIL · INTERPRETATIONS IDENTIFIED IN CONTEXT

Presentation state is validated and stored separately. Hosted writes are owner-authorized and revision-checked; local state is keyed per artifact. The contract prohibits presentation editing from overriding reviewed values and source metadata. The simulated stale-narrative warning here teaches the refresh obligation; it is not a claim that Data automatically detects all stale prose.

Generate the analysis. Reuse the experience.

If every answer generates a new React app from scratch, every answer also reinvents source inspection, editing, filters, and export behavior.

Agent-authored
content

Restricted
authoring contract

Verified
compiler / build

Shared
runtime

src/content/ · authored

```
// Illustrative binding, not a full app
<DataComponent
  id="engagement-rate"
  queryId="engagement-quarterly"
  kind="metric"
  title="Advisor engagement"
>
  {/* reviewed content */}
</DataComponent>
```

Shared experience · maintained once

Source inspection

Charts + common transforms

Presentation editing

View state + component links

Export + host handoff

More predictable artifacts. More framework responsibility.

ARCHITECTURE DETAIL · INTERPRETATIONS IDENTIFIED IN CONTEXT

The ordinary build verifies prebuilt runtime/compiler bytes, then links supported authored modules and local assets. The supported module set includes @openai/data-app, React/React DOM, React Markdown, JSX runtimes, and Recharts. Unsupported package/URL imports, top-level await, and lazy dynamic-import semantics are rejected. Vite belongs to maintainer or explicit source builds. Integrity checks are not a malicious-code sandbox or independent publisher signature.

One evidence foundation. Two ways to think.

Choose the surface from the reader's job, not from whether the input arrived as a spreadsheet.

REPORT / GUIDE THE ARGUMENT

**"Tell me what happened
and what matters."**

Illustrative report composition

Engagement needs a closer look.

The reviewed period is 18.4%. Explain
the comparison, strongest supported
drivers, and limitations here.



Methods & evidence +

Answer-first narrative · selective visuals ·
conclusions with supporting evidence

DASHBOARD / SUPPORT EXPLORATION

**"Let me compare,
filter, and monitor."**

Illustrative dashboard composition

Quarter ▾

Cohort ▾

18.4%

184 / 1,000
same fixture
evidence



Chart placeholders · no measured cohort values

Controls · comparisons · repeatable
exploration · selected-view state

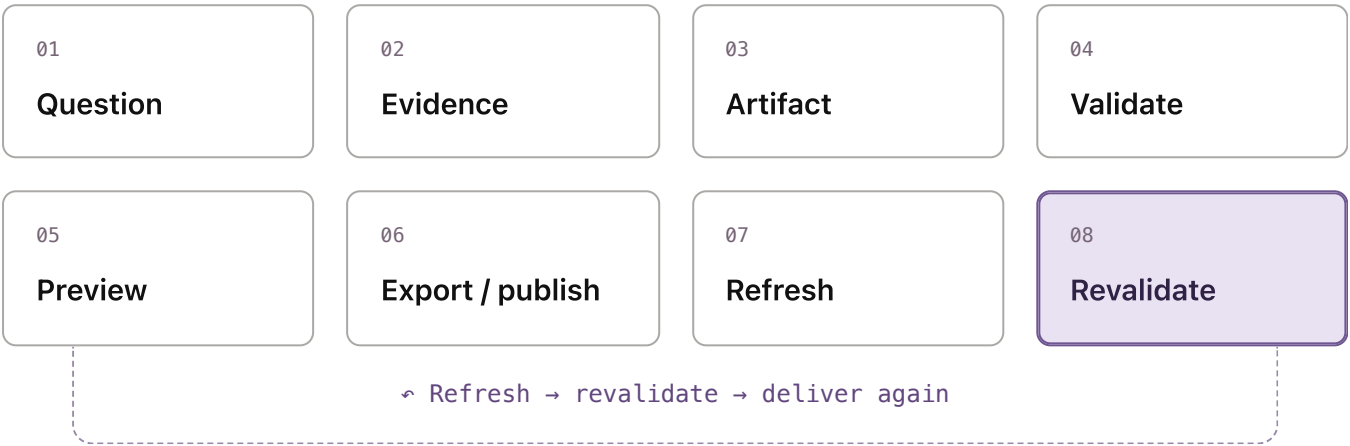
Shared evidence bindings + shared runtime + source inspection

ARCHITECTURE DETAIL · INTERPRETATIONS IDENTIFIED IN CONTEXT

Both surfaces select authored content inside DataAppRuntime / DataAppShell. Report guidance owns the narrative argument; dashboard guidance owns hierarchy, useful views, and scoped controls. The mockups here are teaching compositions, not screenshots of a running Data artifact. Both retain the obligation that visible values, source previews, filtered populations, and exports agree.

The end of one request is the start of a lifecycle.

A saved artifact can be delivered, edited, exported, and refreshed without forgetting what it is.



8. Revalidate

Update affected narrative, periods, and comparisons; verify the new artifact. Return to delivery.

Local preview

Offline export

Sites publication

SITES PUBLICATION

Exact compiled UI + hosted snapshot and presentation

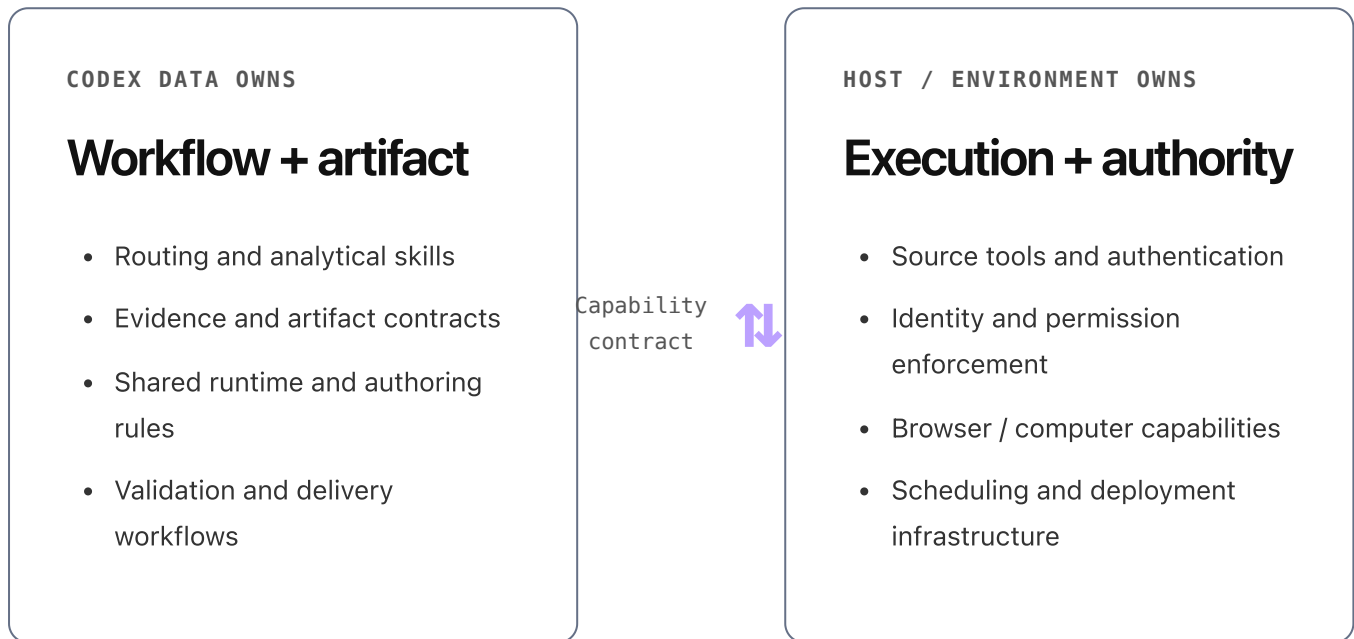
The delivery workflow packages the current app, transfers bound assets, and checks readback while preserving Site access and user state.

ARCHITECTURE DETAIL · INTERPRETATIONS IDENTIFIED IN CONTEXT

Split builds bind HTML to a content-addressed snapshot sidecar. Offline export embeds the verified data without recompiling. Publication packages exact compiled UI with a Worker and hosted data/presentation services: D1 for presentation and revision metadata, R2 for immutable indexed payloads. The current flow deploys before asset upload/readback, which creates a staging trade-off. Refresh reads the current snapshot, presentation, and deployed source; reruns saved requests; rebuilds; preserves identity/access; then checks the result.

The plugin defines the job. The host makes it executable.

Data can reuse different governed tools because it does not embed every database and API client.



Conceptual capability switches

☐ Authorized source access ☐ Sites deployment

Existing snapshots can render, but fresh investigation / refresh needs source access. Local preview and offline export remain possible; Sites publication is unavailable.

— Under the Hood / implementation + sources

ARCHITECTURE DETAIL • INTERPRETATIONS IDENTIFIED IN CONTEXT

The manifest declares skills and optional connector categories, not a plugin-owned analytics MCP server. Actual source execution, browser actions, scheduling, and Sites hosting cross host boundaries. Worker write authorization assumes a trustworthy authenticated identity header supplied by the host. The capability switches summarize available operations; they do not test a live host or change real permissions.

What should we steal?

Borrow the boundaries that make analytical work durable, inspectable, and easier to change.

01 Semantic skills	02 Artifact contract	ENGINEERING TAKEAWAY 9 / 9 Progressive disclosure WHAT DATA DOES Sources and methods remain inspectable behind the main narrative. WHY THIS CAN HELP Readers can start with the answer, then deepen their inspection when needed. WHERE TO USE IT Use in complex analytical products. Keep consequential caveats near the claim, not hidden in a drawer. Inspect the basis ↗
03 Evidence-first authoring	04 Stable identities	
05 Separate state domains	06 Shared experience runtime	
07 Progressive validation	08 Explicit host boundary	
09 Progressive disclosure		

Start with a clear contract. Add machinery when the product needs it.

— Under the Hood / implementation + sources

ARCHITECTURE DETAIL • INTERPRETATIONS IDENTIFIED IN CONTEXT

These are engineering takeaways drawn from the source-backed architecture, not measured claims that adopting them improves every system. Each pattern needs a product need and a validation strategy. The original review retains the detailed implementation findings and prioritized plugin-improvement work.

Learn the principle. Question the machinery.

A sophisticated artifact system solves a sophisticated product problem. A smaller agent may need much less infrastructure.

Explore a trade-off

Inspection discoverability



Inspection discoverability

Deep source controls can be hard for a casual reader to find.

FOR A SMALLER AGENT

One visible “show the basis” action and caveats beside claims.

WHEN THE PRODUCT GROWS

Test whether readers can identify denominator, scope, freshness, and limitations.

Inspect the basis ↗

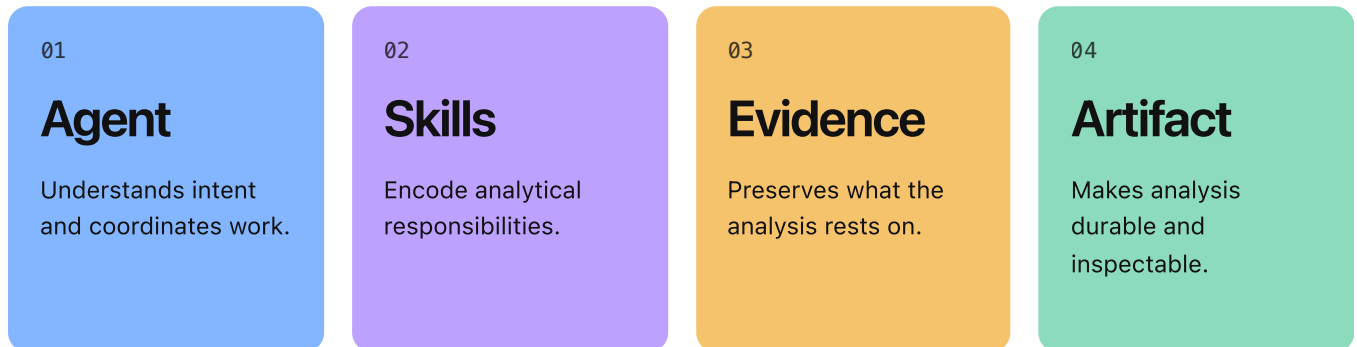
Copy a design because your constraints demand it—not because the source system has it.

ARCHITECTURE DETAIL · INTERPRETATIONS IDENTIFIED IN CONTEXT

The review identifies overlapping context/source-discovery guidance, multiple build and storage paths, rich heterogeneous metadata, and menu-based inspection complexity. Their impact is an architectural interpretation, not a benchmark or usability-study result. The plugin's exact implementation is one point in a design space; its scale of investment is not a prerequisite for every analytical agent.

Agent → Skills → Evidence → Artifact

The model is the same. Now each word carries an architecture.



- 01 Skills encode analytical responsibilities.
- 02 Evidence is preserved, not merely consumed.
- 03 Artifacts have stable contracts and identities.
- 04 Presentation is separated from analytical evidence.
- 05 The host executes capabilities; Data organizes the workflow and artifact experience.

► Check your mental model

Explore the research appendix ↓

— Under the Hood / implementation + sources

ARCHITECTURE DETAIL • INTERPRETATIONS IDENTIFIED IN CONTEXT

The core model is a teaching abstraction of Data 1.0.8. Evidence retention and presentation continuity are concrete contracts; analytical correctness and host permissions remain separate obligations. The appendix preserves the complete standalone review, source ledger, original observations, test receipts, limitations, and improvement proposals.

Keep learning below the surface.

The complete original review is embedded here, including architecture findings, source references, test receipts, limitations, and improvement proposals. Explore it without leaving this file.

Teaching layer → implementation detail → original evidence. The learning exercises are illustrative simulations, not a running Data app.

Based on installed Codex Data 1.0.8 and the standalone review of 14 September 2026. Architectural interpretations and adoption lessons are separated from implementation facts in the deep dives. No live connector, Sites identity flow, or cloud refresh was executed for this lesson. The installed plugin and original review are unchanged.